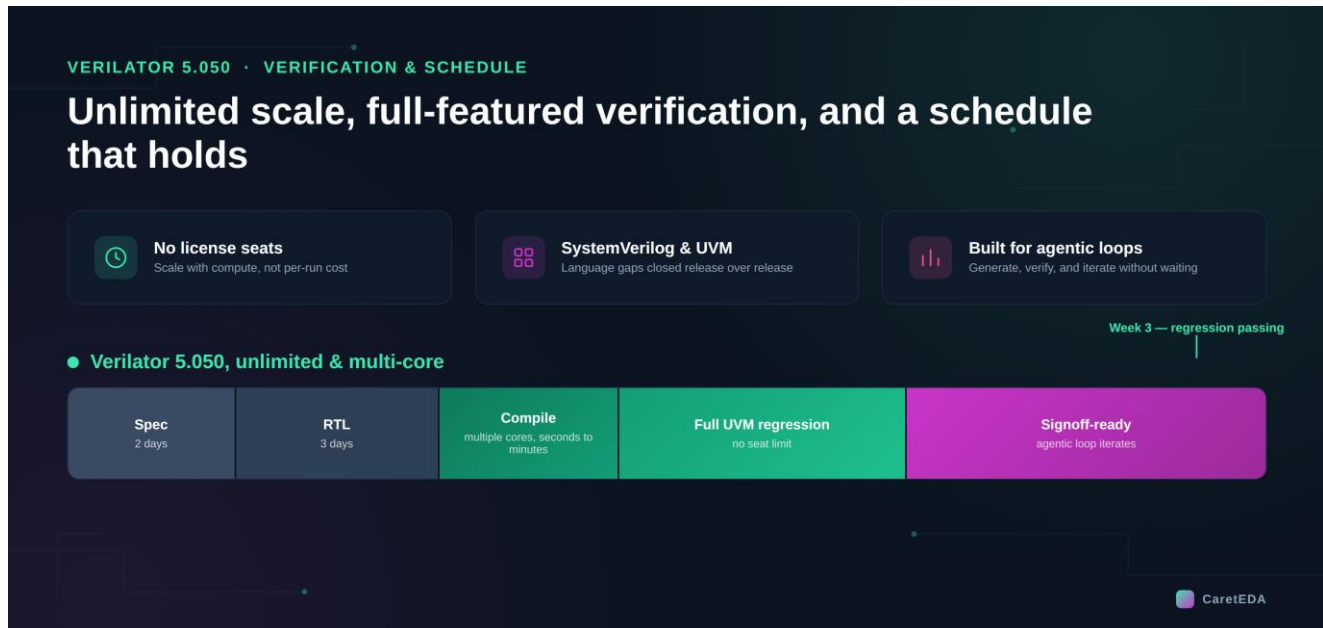


Unlimited. Full-Featured. Perfect for the Agentic Loop.

How Verilator 5.050 closes the cost, functionality, and performance gaps at once — measured across sixteen public and UVM designs

Sashi Obilisetty | Co-founder & CEO, CaretEDA | September 2026



While commercial simulators are fast, and support almost the full subset of SystemVerilog, what they are not is unlimited. Every parallel regression, every branch that wants its own run, every extra job an agentic flow wants to launch competes for a finite pool of seats — and the bill scales upward with how hard you want to push verification, not with how much compute you actually have.

Verilator has always been the opposite bet: no seats, no per-run cost, scale with compute instead of licenses. For a long time that came with a real tax — gaps in the SystemVerilog subset UVM environments depend on, and simulations that stalled out on designs commercial tools handled without complaint. That tax has been shrinking fast. Since the IEEE-compliant scheduler landed in Verilator 5.0, successive releases have closed language gaps one by one, and as of late 2025 Verilator elaborates upstream UVM 2017-1.0 without patches. Version 5.050 is the clearest evidence yet of where that arc has landed.

We didn't take that on faith. We ran sixteen public designs (some with UVM testbenches) through Verilator 5.050, compared eight of them directly against 5.042, and tracked what changed when compilation moved from a single core to eight. The results answer the question that actually matters for an agentic verification loop: **can you get unlimited, full-featured, and fast in the same simulator, or is that still three separate simulators?**

Multi-core compilation turns hour-long builds into a coffee break

Verilator translates RTL into C++ and compiles it into a simulator binary, and that compilation step — not simulation itself — was the long pole for our larger UVM designs. Spreading that build across eight cores with **--build-jobs** cut single-core compile times by 17x to 59x on our three largest UVM environments, turning hour-long builds into single-digit minutes regardless of design size. Table 1 has the full breakdown.

Design	Single-core compile	8-core compile	Speedup
VeeR EL2 (UVM)	51m 55s	1m 0.4s	52×
Ethernet MAC (UVM)	70m 11s	1m 11s	59×
Coral NPU	17m 6s	1m 1s	17×

Table 1. Compile time, single core vs. eight cores (--build-jobs), Verilator 5.050.

The range matters more than any single number. Multi-core compilation isn't a trick that only pays off on toy designs — it scales down the wait for a 4,600-line Ethernet MAC and a 470,000-line NPU alike, just by different factors depending on how parallelizable the generated C++ is. For a team running regression multiple times a day, that's the difference between a build you wait out and a build you don't notice.

Designs that used to fail outright now simulate

Speed only matters if the design compiles in the first place, and that was the harder gap to close. We compared eight designs across the jump from 5.042 to 5.050 — on 5.042, more than half failed to compile or complete simulation; on 5.050, every one of them built and ran clean. Table 2 breaks down each design.

OpenPiton and Caliptra-RTL are the sharpest examples: both went from a hard compile failure on 5.042 to a completed simulation run on 5.050, on designs Verilator flatly could not touch six releases ago. VeeR EL2's UVM environment, AHB-APB VIP, and the Ethernet MAC UVM testbench compiled on both versions but only started passing simulation on 5.050.

Design	Status on 5.042	5.050 compile time	5.050 simulation
OpenPiton	Failed to compile	1m 58s	Successful
Caliptra-RTL	Failed to compile	3m 53s	Successful
Processor-UVM	Failed to compile	1m 8.75s	Successful
SDRAM (UVM)	Failed to compile	7.743s	Successful
VeeR EL2 (UVM)	Compiled, simulation failed	1m 14.84s	Successful
AHB-APB VIP (UVM)	Compiled, simulation failed	1m 22s	Successful
Ethernet MAC (UVM)	Compiled, simulation failed	1m 11s	Successful

Table 2. Designs that failed on Verilator 5.042, compiling and simulating on 5.050.

We're not claiming these runs constitute complete verification — a passing simulation confirms a design ran, not that every test passed or that coverage is sufficient. What it confirms is that the tool now gets out of the way. An engineer working against these designs used to start the day with a build log. Now they start with a waveform.

UVM at a scale that isn't a toy testbench

The gap that has kept Verilator out of some production verification flows was never raw speed — it was whether it could carry a real UVM environment. We pushed on that directly with Coral NPU: 471,639 lines of DUT code, 19,680 lines of UVM environment, 613 elaborated modules, and a 764-test regression across two test classes. It compiled, elaborated, and ran the full regression on Verilator 5.050.

That's not an outlier we're hiding behind. SDRAM ran a 50-test regression clean. The Ethernet MAC UVM testbench elaborated 425 modules across 2,647 generated C++ files and passed. AHB-APB VIP, with 1,463 lines of UVM environment against a 491-line DUT, completed its full regression in 83 seconds on eight cores.

Verilator new functionality development is on hyperdrive to address pending gaps. Caliptra-RTL's UVM environment needs commercial VIPs we don't have license to run. This is real engineering progress against real designs, not a curated highlight reel.

An open source stack CaretEDA stands behind

None of this — faster builds, designs that now compile, UVM environments that now run — removes the work of tracking Verilator's fast-moving development, validating a given version against your design, or having someone to call when a regression breaks three weeks before tapeout. That's the layer CaretEDA adds. Our 2026.08 release ships Verilator 5.050 as part of a commercially supported open source EDA stack — alongside Yosys, SymbiYosys, and OpenROAD — with SLA-backed response commitments and platform-specific packages your team doesn't have to assemble by hand.

The reason this matters beyond Verilator on its own: Verilator 5.050 sits directly inside our Spec-to-Netlist™ platform's agentic flow. The gains we measured here aren't a separate benchmarking exercise; they're the verification stage every design generated through CaretEDA runs through.

An agentic design loop lives or dies on how fast it can iterate: generate, verify, learn, regenerate. That loop can't wait in a license queue between cycles, and it can't afford a simulator that stumbles on the UVM environment the design actually needs. Unlimited, full-featured, and fast used to be a pick-two problem. Verilator 5.050 is where that stops being true — and CaretEDA gives you a partner accountable for keeping the whole stack running underneath it.

Benchmark and source notes

Compile and simulation timings are from a benchmark set spanning sixteen public and UVM designs on Verilator 5.050, with an eight-design comparison against 5.042. Multi-core figures use Verilator's --build-jobs option on eight cores. A recorded simulation run confirms the design executed, not complete functional verification or full coverage closure.

About CaretEDA

CaretEDA builds AI-native design systems for modern semiconductor teams, with its Spec-to-Netlist™ platform serving as a powerful force multiplier across the entire engineering workflow. To learn more, apply to the Startup Program, or request a demo, visit www.CaretEDA.com.